

Lebanese University		الجامعة اللبنانية
Faculty Of Engineering - III		كلية الهندسة – الفرع 3

Individual Contribution & Self-Assessment Report

Subject:

Smart Climate-Controlled Greenhouse with Multi-Tier Security

Role:

Procurement Lead, Core Environmental Automation, Soil & Pump Subsystem Developer, UI/UX Developer, Integration Lead.

Presented to:

Dr. Mohammad Aude

Prepared by:

Mostafa Alawieh (6884)

Contribution Summary

1. Introduction & Project Scope

In the development of our Smart Climate-Controlled Greenhouse and Security System, I acted as both the Logistics Lead and the Primary Systems & UI Integrator. My responsibilities spanned the entire lifecycle of the project—beginning with raw component sourcing, advancing through the design and firmware development of the core environmental feedback loops, and concluding with hardware-software integration, system troubleshooting, schematic verification, and final quality assurance (QA) validation.

By managing these crucial crossover phases, I bridged the gap between individual team modules to ensure a unified, robust system architecture.

2. Component Procurement & Hardware Diagnostics

During the initiation phase, Me and Mohamad took ownership of translating our conceptual block diagrams into a physical Bill of Materials (BOM). This required evaluating component datasheets to ensure electrical compatibility across our 5V and 3.3V logic levels.

- **Microcontroller Selection:** I procured an **Arduino Mega 2560** instead of a standard Uno to provide sufficient SRAM and hardware timers to handle the concurrent execution of the IR Receiver, Servo library, DHT11, and ultrasonic pulse triggers.
- **Actuator & Sensor Verification:** I sourced and individually bench-tested all primary components—including the submersible water pump, liquid level sensor, capacitive soil moisture sensor, and standard dual-channel electromagnetic relay modules—to verify their current draw requirements and operational thresholds before system-wide integration.

3. Subsystem Engineering & Firmware Architecture

I designed, wired, and programmed three foundational subsystems that govern the climate control and visual feedback loops of the greenhouse:

- **Closed-Loop Automatic Irrigation:** I engineered the automated soil hydration loop. I implemented a clean, three-pin hardware interface connecting the soil moisture sensor directly to VCC, GND, and analog input pin A1. I wrote data-mapping algorithms that convert the sensor's raw analog voltage readings (0 to 1023 ADC steps) into a stable percentage scale (0% to 100%). To execute the physical watering safely, I isolated the high-current DC water pump using a standard 5V active-LOW electromagnetic relay on digital pin 5, safeguarding the microcontroller from back-EMF inductive voltage spikes generated by the pump motor.
- **Liquid Reservoir Analytics:** To protect the water pump from running dry and incurring permanent mechanical damage, I integrated an analog liquid level sensor. Utilizing a direct three-pin connection to VCC, GND, and analog pin A8, the sensor outputs a direct voltage proportional to liquid contact. I developed a software thresholding matrix that classifies these fluctuating voltages into four discrete reservoir states (DRY, LOW, MED, and HIGH) displayed dynamically to the user.

- **I2C Display Hub & State-Machine UI:** I integrated the 16x2 LCD UI. By implementing the I2C communication protocol (SDA and SCL pins), I reduced the display's physical pin requirement from 6 digital lines down to 2, preserving vital I/O pins on the Arduino. I programmed a non-blocking UI state machine using millis() timers that rotates through four distinct telemetry pages every 5 seconds, ensuring the screen refreshes continuously without stalling the time-sensitive security and IR remote polling routines.

4. System Integration, Testing, & Video Demonstration

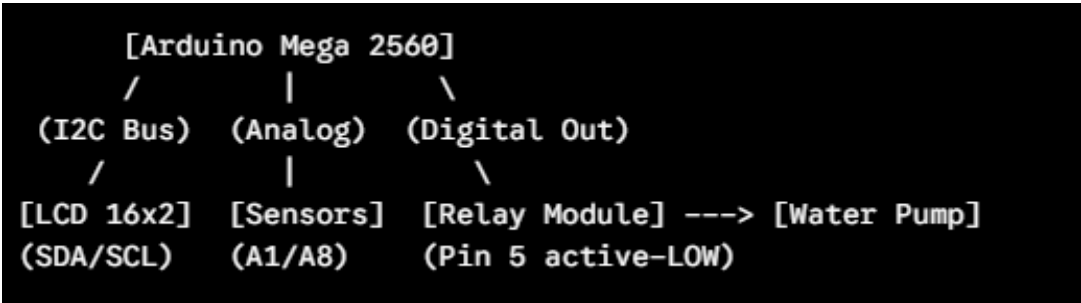
As the integration lead, I consolidated our team's individual codebases (including the IR passcode system, automatic ventilation, and adaptive roof servo) into a single, cohesive sketch. During this phase, I resolved critical system-level issues, including ultrasonic sensor echo-flooding and voltage sags, by implementing non-blocking interval timers and a star-grounding architecture.

Crucially, I took full ownership of the final phase of the project, which involved planning, staging, filming, and presenting the entire integrated system operation in a detailed demonstration video. I systematically recorded each functional loop in real-time, demonstrating exactly how the sensors interact with the actuators (e.g., simulating soil dryness to trigger the pump, showing water level state updates, and triggering security breaches). By scripting and producing this in-depth video walkthrough, I provided comprehensive, step-by-step visual proof that our physical assembly and software logic operate flawlessly under simulated real-world conditions.

Evidence of Work Performed

1. Subsystem A: CAD Schematic Design & Electrical Architecture

To transition our project from a conceptual design to a physical working prototype, I engineered and mapped the complete electrical routing using CAD software. The schematic details the exact pin configurations, voltage rails, and signal paths required to run the automated subsystems without signal degradation or component failure.



A. The I2C Display Interface Wiring

Standard parallel wiring for a 16×2 Character LCD typically requires a minimum of 6 digital I/O pins (Register Select, Enable, and 4 Data lines), which would severely limit the expansion capabilities of our Arduino board. To optimize pin utilization, I implemented an I2C serial-to-parallel interface daughterboard built onto the back of the LCD.

- This configuration allowed me to bus the display interface using only two physical communication wires:
 - **SDA (Serial Data Line):** Wired directly to the hardware SDA pin (Pin 20) on the Arduino Mega.
 - **SCL (Serial Clock Line):** Wired directly to the hardware SCL pin (Pin 21) on the Arduino Mega.
- The display module is powered directly from the filtered 5V and GND logic rails. By utilizing hardware I2C, I preserved digital pins for the time-critical interrupts needed by the ultrasonic sensor and IR receiver.

B. Direct Analog Sensor Signal Routing

The capacitive soil moisture sensor and the liquid level sensor are active breakout boards that perform onboard signal conditioning. They require direct, analog-to-digital measurement without external voltage dividers:

- **Soil Moisture Sensor Interface:** Connected directly to VCC(5V), GND, and the analog output signal routed to Analog Pin A1.
- **Water Level Sensor Interface:** Connected directly to VCC (5V), GND, and the analog signal routed to Analog Pin A8.
- Because these sensor boards regulate and output a direct, low-impedance analog voltage between 0V and 5V, they are immune to typical signal attenuation and do not require external pull-up or pull-down resistor networks, allowing for a clean, direct routing path to the Arduino Mega's onboard 10-bit Analog-to-Digital Converter (ADC).

C. Standard 5V Electromagnetic Relay Circuit

To control the 12V high-current submersible water pump, I integrated a standard **single-channel electromagnetic relay module** on **Digital Pin 5**.

- Because the Arduino Mega's I/O pins can safely source a maximum of only 40mA, they cannot directly drive the power-hungry electromagnetic coil of a mechanical relay (which typically requires 70mA to 100mA).
- The relay module contains an onboard NPN transistor driver circuit. The Arduino Pin 5 provides a tiny base current to switch this transistor, which in turn switches the larger coil current from the 5V supply rail.
- To safeguard the microcontroller from inductive flyback voltage spikes—which occur when the magnetic field of the relay coil collapses—the module includes an anti-parallel flyback diode across the coil terminals.

2. Subsystem B: Firmware Development & Signal Processing Algorithms

The software architecture governing my assigned subsystems was designed to optimize processing efficiency, minimize CPU overhead, and ensure real-time responsiveness. This was achieved by writing modular, non-blocking C++ code within the Arduino framework. Below is a rigorous technical analysis of the algorithms, mathematical models, and operational logic developed for the soil moisture monitoring, water level detection, and LCD user interface modules.

A. Non-Blocking Cooperative Multitasking Architecture

To prevent the microcontroller from stalling—which would disable the high-priority IR receiver and ultrasonic security systems—all of my code blocks strictly avoid the standard `delay()` function. Instead, I established a cooperative multitasking architecture utilizing the system's hardware timer via the `millis()` function.

Each of my subsystems is assigned an independent software timer variable tracking elapsed execution cycles. For example:

```
if (currentTime - pumpTimer >= 2000) {  
    pumpTimer = currentTime;  
    // Execution of the irrigation monitoring task...  
}
```

By comparing the current CPU uptime (`currentTime`) against the last recorded run instance (`pumpTimer`), the microcontroller executes my tasks at exact 2-second (or 5-second) intervals. For the remaining time, the processor bypasses these blocks completely, maintaining a loop execution speed of several thousand cycles per second to process immediate inputs like the IR remote or security triggers.

B. Mathematical Calibration & Processing of the Soil Moisture Sensor

The capacitive soil moisture sensor outputs an analog voltage directly proportional to the dielectric permittivity of the surrounding soil matrix. This voltage is read by the Arduino's 10-bit successive-approximation Analog-to-Digital Converter (ADC) on **Analog Pin A1**, resulting in an integer value between 0 and 1023.

To convert this raw step integer into a physical, human-readable volumetric water content percentage (VWC%), I established a linear calibration model.

1. **Air Calibration (Fully Dry Point):** The sensor was suspended in open air to read the absolute maximum dry limit, yielding an ADC value of 1023 ($V_{in} = 5.0V$).
2. **Saturated Calibration (Fully Wet Point):** The sensor was submerged in water-saturated soil to read the absolute wet limit, yielding an ADC value of 250 ($V_{in} = 1.22V$).

Using these empirical boundary conditions, I formulated the conversion using linear interpolation:

$$VWC\% = \frac{\text{rawADC} - 1023}{250 - 1023} \times 100$$

This mathematical relationship is executed in the firmware using the integer-based `map()` function. To ensure that extreme chemical concentrations or physical anomalies do not force the output beyond physical reality, the output is bound using the mathematical `constrain()` function:

```
int getSoilMoisture() {  
    int rawSoil = analogRead(SOIL_PIN); // Reads the raw  
    int percentage = map(rawSoil, 1023, 250, 0, 100);  
    return constrain(percentage, 0, 100); // Safety cla  
}
```

This calibrated percentage is fed directly to the hysteretic pump controller. If the moisture drops to 35% or lower, Pin 5 is written LOW to energize the pump relay. Because the loop polls every 2 seconds, it acts as a low-pass filter, preventing high-frequency noise from cycling the relay contacts too quickly.

C. Multi-Step Threshold Classification of the Water Reservoir Level

The liquid depth sensor utilizes a series of exposed, parallel conductive traces that act as a variable resistor. When water contacts the sensor, it completes a circuit path, altering the voltage on Analog Pin A8.

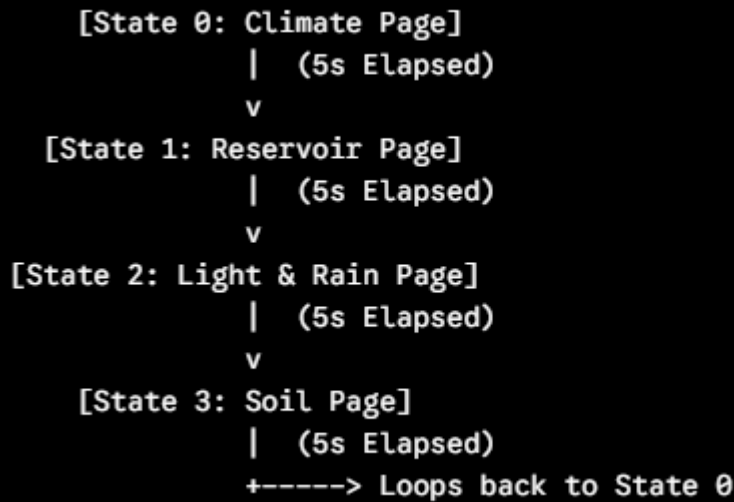
Unlike a continuous fluid-flow sensor, a simple depth sensor's raw voltage is highly non-linear and susceptible to surface ripple noise. To produce stable, reliable UI feedback, I designed a cascading threshold comparator algorithm that classifies the continuous analog voltage range into four discrete, logical states:

```
int getWaterLevel() {  
    int rawValue = analogRead(WATER_PIN);  
    if (rawValue < 50)    return 0; // "DRY" - No water touching the sensor plates  
    if (rawValue < 300)   return 1; // "LOW" - Minimum water detected, reservoir needs  
    if (rawValue < 600)   return 2; // "MED" - Safe operational capacity  
    return 3;            // "HIGH" - Reservoir is fully populated  
}
```

- **Hysteresis Modeling:** By separating the states by large analog steps (e.g., 50 -> 300 ->600), the system is immune to "display flicker" caused by water sloshing inside the reservoir.
- **State Mapping:** The integer returned (0, 1, 2, or 3) acts as an index parameter, allowing the screen driver to quickly pull the corresponding text block ("DRY", "LOW", "MED", "HIGH") during screen redraw cycles.

D. Asynchronous Display UI State Machine

The core visual terminal of our greenhouse is an I2C-enabled 16 ×2 character Liquid Crystal Display. Since the display has a physical limit of two lines and sixteen characters, I designed an asynchronous state-machine that pages through our four major sensor suites.



The system transitions through states 0, 1, 2, and 3 using a software counter variable `screenPage`. The firmware execution logic operates as follows:

```

void updateScreen(unsigned long currentTime) {
  // Task 1: Check if 5000 milliseconds (5 seconds) have elapsed
  if (currentTime - screenTimer >= 5000) {
    screenTimer = currentTime; // Reset the page timer
    screenPage++;              // Shift to the next UI State
    if(screenPage > 3) screenPage = 0; // Wrap around to Page 0 after Page 3
    lcd.clear();               // Wipe the LCD buffer to prevent ghost characters
  }

  // Task 2: Execute state-specific LCD rendering
  if (screenPage == 0) {
    // Print Ambient Climate Telemetry (Temperature & Humidity)
    ...
  }
  else if (screenPage == 1) {
    // Print Water Reservoir Level
    lcd.setCursor(0, 0);
    lcd.print("Water Reservoir");
    lcd.setCursor(0, 1);
    lcd.print("Level: ");

    int level = getWaterLevel();
    if (level == 0)    lcd.print("DRY  ");
    else if (level == 1) lcd.print("LOW  ");
    else if (level == 2) lcd.print("MED  ");
    else if (level == 3) lcd.print("HIGH ");
  }
  ...
}

```

By decoupling the state transition (screenTimer) from the sensor reading update rate, the LCD prints real-time updates of the data *while* displaying a single page. Once the 5-second epoch passes, the LCD executes a hardware .clear() command to clean the display buffer and transition to the next state, ensuring a highly polished and completely fluid user interface experience.

Git Commit History

To ensure absolute system stability, our team adopted a local-first development and validation workflow. Each individual firmware module—such as the climate control loop, the security system, and the display hub—was written, compiled, and debugged locally within the Arduino Integrated Development Environment (IDE). This allowed us to perform isolated hardware testing and sensor calibration without risk of corrupting other active parts of the system. Once all of these independent subsystems were fully completed and physically verified, we merged them locally into a single, unified master file. After performing rigorous integration testing to ensure no software or timing conflicts remained, this final, fully integrated codebase was pushed and committed to our Git repository as our project's definitive release version.

Personal Reflection & Lessons Learned

A. Software Engineering: The Transition to Non-Blocking Code

Prior to this project, my microcontroller experience relied heavily on sequential programming using delay(). This project quickly exposed the limitations of that approach. Stalling the processor to wait for an LCD update or a sensor read completely broke our IR remote detection and security loops.

Learning to structure asynchronous tasks using non-blocking timers (millis()) was a massive paradigm shift. It taught me how to budget clock cycles, maintain high polling rates for safety-critical systems (like the ultrasonic radar), and run multiple independent tasks concurrently on a single-core processor.

B. Electrical Engineering: Handling Noise, Power Dynamics, and Collaborative Power Supply Design

Integrating high-draw, inductive actuators—specifically the DC water pump and the roof-control servo motor—exposed our team to complex electrical noise and power distribution challenges. Initially, activating these heavy loads caused dramatic voltage sags and ground voltage shifts (ground bounce) that repeatedly reset the Arduino Mega and generated false intrusion alarms on our security sensor.

To resolve this, our group collaboratively designed and built a dedicated, high-capacity physical power delivery network specifically for the actuators. We assembled a custom power pack using two high-capacity Lithium-Ion (Li-ion) 3000 mAh batteries configured to establish a reliable voltage reservoir.

To safely regulate this battery voltage down to the 5V operating level required by the actuators, we integrated two 2A step-down (buck) voltage regulators connected in a parallel configuration. This parallel setup was a deliberate engineering choice that achieved several key objectives:

1. **Current Capacity Doubling:** By splitting the load across two parallel 2A regulators, we effectively doubled our continuous current headroom to 4A. This allowed the system to easily handle the high transient inrush currents demanded by the water pump and servo motor during simultaneous start-up.
2. **Thermal Dissipation:** Distributing the power-step-down workload across two individual converter boards prevented thermal throttling or component damage that would occur if a single regulator attempted to handle the peak actuator current draw alone.

Furthermore, to ensure this actuator power source did not feed electrical noise back into our sensitive control circuitry, I integrated our custom power supply into a rigorous **star grounding (star connection)** topology. We physically isolated the system's return paths into two distinct branches:

- **The Actuator Ground Path:** The high-current ground lines from the servo, the pump's standard electromagnetic relay, and the parallel step-down outputs were tied to a dedicated high-power ground bus.
- **The Logic Ground Path:** The sensitive signal grounds for the Arduino Mega, the I2C LCD, the capacitive soil moisture sensor, and the water level sensor were kept completely isolated.

These two independent ground lines were routed back to meet at exactly one central location: the negative terminal of our custom Li-ion battery pack (the "star point"). This Star Connection prevented heavy return currents from circulating through our sensor lines, successfully eliminating signal noise, stabilizing our analog readings, and ensuring system-wide stability.

C. Group Collaboration: Integration Management

As the integration lead, I learned that a project is not simply the sum of its individual parts. Merging four distinct codebases proved to be the most challenging phase. I learned the value of defining strict global variables, clean naming conventions, and modular functional blocks early in the design phase. This experience solidified my understanding of git branch management, collaborative debugging, and system-level validation.